User-Defined Static Property Tags for Mainstream Programming Languages: A Lightweight Flow-Sensitive Verification Layer

Sambuddha Majumder, Jayanta Majumder

August 30, 2026

Abstract

This paper proposes a lightweight static verification mechanism for mainstream programming languages in which program values may be associated, at compile time, with user-defined symbolic *property tags*. A tag records a fact believed or proven about a value, such as `Validated`, `BoundsChecked`, `Authenticated`, `SQLSafe`, or `DeltaNeutral`. Programmers may introduce such properties at trusted proof points, require them at safety-critical program points, and specify interprocedural contracts describing how functions consume, preserve, invalidate, or establish properties. A flow-sensitive static checker propagates these facts over the control-flow and data-flow graphs and rejects code whenever a required property is not provable on every incoming path. The proposal intentionally occupies a design point between conventional static analysis and substantially more expressive systems such as refinement types, typestate, effect systems, abstract interpretation frameworks, and dependent type theory. Its principal aim is to expose correctness-relevant semantic state without requiring programmers to encode full logical predicates or construct explicit proofs. We present a core semantics, discuss positive, negative, and unknown property states, outline intraprocedural and interprocedural analyses, examine aliasing and mutation, propose syntax and contract forms, and analyse soundness, usability, expressiveness, and implementation strategies. Several extensions are considered, including parametric tags, capability-like tags, tag lattices, proof-producing validators, trust boundaries, generic tag transformers, and gradual adoption in existing languages.

1 Introduction

Mainstream programming languages provide increasingly strong static guarantees about the *shape* of values: their data types, nullability, generic parameters, ownership status, mutability, and in some cases lifetime relationships. Many important software correctness properties, however, are not naturally captured by conventional nominal or structural types. A string may be syntactically a `String` yet differ critically depending on whether it has been validated, normalised, authenticated, escaped for a particular output context, checked against an access-control policy, or derived from a trusted source. Similarly, a numeric quantity may be known to be range checked, calibrated, monotonic, hedged, normalised, or otherwise semantically constrained even though its ordinary machine type remains unchanged.

Developers commonly encode such information informally in comments, naming conventions, call discipline, wrapper classes, assertions, or documentation. Static analysis tools may recover some of these facts automatically, but such analyses are generally specialised and are not always exposed as a general-purpose language abstraction. More expressive type systems can model many of these properties, but they may impose conceptual, syntactic, or proof burdens that inhibit adoption in large mainstream codebases.

This paper proposes a language-level mechanism based on *user-defined static property tags*. Conceptually, a programmer may write:

```
establish x Validated;
require   x Validated;
```

where `Validated` is a symbolic property that exists entirely at compile time. A checker propagates the property along value flows. Reassignment to a different value does not preserve the property unless justified; copying a value may preserve it; transformations may preserve, drop, or establish properties according to declared contracts; and control-flow joins retain only those facts guaranteed on all incoming paths.

The central design principle is that tags attach to *values and value provenance*, rather than merely to lexical variable names. The system therefore resembles a lightweight proof environment over an intermediate representation such as static single assignment (SSA) form.

The proposal has four goals:

1. **Low annotation burden.** Programmers should be able to introduce useful semantic facts without learning a full theorem prover or dependent type discipline.
2. **Flow sensitivity.** Properties should vary naturally across branches, loops, assignments, and calls.
3. **Interprocedural compositionality.** Functions should state what properties they require and establish.
4. **Compatibility with mainstream implementation practice.** The mechanism should be implementable as a front-end analysis, compiler plug-in, lint phase, or optional language extension.

2 Motivating Example

Consider data received from an untrusted source:

```
String input = request.parameter("name");

if (!validateName(input))
    return error();

establish input Validated;

String escaped = sqlEscape(input);
establish escaped SQLSafe;

executeQuery(escaped);
```

A library contract might specify:

```
function executeQuery(String q)
    requires q SQLSafe;
```

The call is accepted only if the checker can prove that the actual argument carries `SQLSafe`. A later reassignment prevents accidental reuse of the earlier fact:

```
establish input Validated;
input = request.parameter("other");
require input Validated;     // static error
```

The important distinction is that the identifier `input` has not permanently acquired the tag. Rather, one specific value denoted by `input` was established to have the property.

3 Core Model

3.1 Property universe

Let $\mathcal{T}$ be a finite or countably extensible universe of symbolic tags. Tags are uninterpreted names at the core language level:

$$\mathcal{T} = \{\mathsf{Validated}, \mathsf{SQLSafe}, \mathsf{Authenticated}, \ldots\}.$$

The core checker need not understand the domain meaning of a tag. It reasons only about property introduction, removal, flow, and contracts.

3.2 Three-valued knowledge

For each value v and tag t, the checker tracks one of three knowledge states:

$$K(v, t) \in \{\mathsf{Present}, \mathsf{Absent}, \mathsf{Unknown}\}.$$

This distinction is essential. Failure to prove that a tag is present is not equivalent to proving that it is absent.

For example:

```
if (condition) {
    establish x Trusted;
}
```

After the branch, `Trusted(x)` is generally *unknown*; neither its presence nor its absence is guaranteed.

The knowledge domain for one tag can be represented as a small lattice in which conflicting path information joins to Unknown. An implementation may equivalently maintain two sets per abstract value: tags definitely present and tags definitely absent.

3.3 Program state

For exposition, let a static analysis state be

$$\Sigma : V \to (P^+, P^-),$$

where V is the set of abstract values, $P^+ \subseteq \mathcal{T}$ contains definitely present tags, and $P^- \subseteq \mathcal{T}$ contains definitely absent tags, with $P^+ \cap P^- = \emptyset$.

A requirement `require x T` succeeds iff $T \in P^+(x)$. A negative requirement `require x !T` succeeds iff $T \in P^-(x)$.

3.4 Join at control-flow merge points

At a merge point with predecessor states $\Sigma_1, \ldots, \Sigma_n$, only universally guaranteed facts survive:

$$P^+_{\text{join}}(v) = \bigcap_i P^+_i(v), \qquad P^-_{\text{join}}(v) = \bigcap_i P^-_i(v).$$

Thus the mechanism is intentionally conservative.

4 Proposed Surface Syntax

A minimal syntax could contain the following forms:

```
establish x Validated;
refute    x Tainted;
require   x Validated SQLSafe;
require   x !Tainted;
forget    x Validated;
assume    x Trusted;
```

The distinction between `establish` and `assume` is important. A language should avoid making an arbitrary programmer annotation appear equivalent to a proved property. Suggested meanings are:

- `establish`: legal only at a checker-recognised proof point, validator success branch, compiler intrinsic, verified contract boundary, or other sanctioned mechanism;
- `assume`: an explicit trust escape hatch analogous to an unchecked cast;
- `require`: compile-time proof obligation;
- `refute`: establish definite absence of a tag;
- `forget`: deliberately discard information, useful around abstraction boundaries or mutation.

A simpler prototype could collapse `establish` and `assume` into a single `tag` operation while producing warnings or trust annotations.

5 Value Identity and SSA Interpretation

The semantics are clearest in SSA form. Consider:

```
x = read();
establish x Sanitized;
y = x;
x = read();
```

Conceptually the compiler may rewrite this as

$$\begin{aligned} x_1 &= \mathrm{read}(), \\ &\quad \mathsf{Sanitized}(x_1), \\ y_1 &= x_1, \\ x_2 &= \mathrm{read}(). \end{aligned}$$

Hence y_1 may inherit Sanitized, while x_2 does not. This value-centric interpretation eliminates a major class of ambiguities inherent in variable-centric tagging.

6 Transfer Functions

Each instruction induces a transfer function over the abstract property state.

6.1 Copy

For immutable scalar values or other copy-semantics values:

$$y := x \implies P^+(y) \supseteq P^+(x), \quad P^-(y) \supseteq P^-(x).$$

A conservative implementation may copy all properties unless individual tags are declared non-transferable.

6.2 Fresh assignment

If x receives a newly computed value, properties attached to its previous value do not automatically transfer.

6.3 Pure transformations

For a function f, the checker may know a contract of the form

$$\{T_1, \ldots, T_k\} \xrightarrow{f} \{U_1, \ldots, U_m\}.$$

For example:

```
contract sqlEscape(x)
    ensures result SQLSafe;
```

or

```
contract normalize(x)
    preserves x Validated;
    ensures result Normalized;
```

6.4 Invalidating transformations

Some operations should explicitly invalidate properties:

```
contract mutateBuffer(b)
    invalidates b Parsed ChecksumVerified;
```

This is particularly important when a property depends on exact representation rather than abstract identity.

7 Function Contracts

Interprocedural verification is central to practical usefulness. We propose contracts such as:

```
function sendPayment(Payment p)
    requires p Validated Authorised;

function verifyPayment(Payment p)
    ensures p Validated;

function authorise(Payment p, User u)
    requires u Authenticated;
    ensures p Authorised;
```

At a call site the checker verifies each precondition from the incoming abstract state and applies the callee's postcondition to the outgoing state. Unknown or unavailable bodies can still participate through interface contracts.

This design supports separate compilation and allows libraries to expose semantic safety requirements without exposing implementations.

8 Path-Sensitive Establishment

Validation APIs often naturally refine program state by branch. For example:

```
if (isValid(x)) {
    establish x Validated;
    consume(x);
}
```

A more ergonomic system may allow the validator's contract itself to refine the true branch:

```
function isValid(x)
    returns true => ensures x Validated;
    returns false => ensures x !Validated;
```

This resembles type narrowing in contemporary languages, but applies to arbitrary user-defined semantic properties.

The following program is then rejected:

```
if (isValid(x)) {
    log("valid");
}
require x Validated;       // not true on all paths
```

whereas this program is accepted:

```
if (!isValid(x))
    return;

require x Validated;       // proven
```

9 Loops and Fixed-Point Analysis

Loops require conventional iterative data-flow analysis. The checker computes an invariant at the loop header as a fixed point. A property is guaranteed after a loop only if justified by the loop semantics and all exits.

For example:

```
while (moreChunks()) {
    readChunk(buffer);
    // validation of earlier contents may no longer be valid
}
require buffer Validated;
```

will generally fail unless the loop invariant or function contracts establish that `Validated` survives every iteration.

An optional advanced syntax could permit programmer-supplied property invariants:

```
while (condition)
    invariant buffer BoundsChecked
{
    ...
}
```

The checker would then verify preservation of the stated invariant.

10 Aliasing, Mutation, and Heap Objects

Aliasing is one of the principal difficulties in any serious implementation. If two references denote the same mutable object,

```
a = object;
b = a;
establish a Validated;
mutate(b);
require a Validated;
```

then the final requirement is sound only if `mutate` is known to preserve `Validated`. Consequently, tags over mutable heap objects cannot be handled solely as lexical annotations.

Several implementation strategies are possible:

1. **Alias-sensitive invalidation.** Use points-to/alias analysis and invalidate affected properties on all may-alias references.
2. **Ownership integration.** Preserve stronger facts when an object is uniquely owned or borrowed immutably.
3. **Versioned heap abstractions.** Treat mutation as creating a new abstract object version, analogous to SSA for memory.
4. **Conservative forgetting.** Drop all representation-sensitive tags after unknown mutating calls.

A production design may combine these approaches depending on compiler sophistication.

11 Tag Classes and Preservation Policies

Not all tags should propagate identically. We propose optional metadata describing tag behaviour.

Tag class	Intended behaviour
Value property	Follows immutable value copies; lost on recomputation.
Representation property	Invalidated by representation-changing transformations.
Object-state property	Associated with mutable object state and sensitive to aliasing.
Capability property	May be affine or linear; copying can be restricted.
Provenance property	Records source or construction history.
Contextual property	Valid only relative to a parameter, principal, encoding, locale, or policy.

The core proposal can remain deliberately simpler, but these classes provide a path to richer semantics.

12 Parametric Tags

User-named boolean tags are useful but sometimes too coarse. A natural extension is parameterised tags:

```
establish s EscapedFor(HTML);
establish amount Range(0, 1000000);
establish token AuthorisedFor(account42);
establish buffer CheckedLength(n);
```

These approach refinement types, but a system may still treat parameters syntactically rather than invoke a general theorem prover. Equality, substitution, and a small set of built-in relations could deliver substantial value without exposing arbitrary logic.

For security-sensitive programs, contextual parameterisation is particularly important. A string escaped for HTML is not necessarily safe for SQL; an authorisation for one resource should not silently generalise to another.

13 Derived and Hierarchical Tags

A tag relation may define implication:

$$\mathsf{CryptographicallyVerified} \Rightarrow \mathsf{IntegrityChecked}.$$

A declaration might be:

```
tag CryptographicallyVerified implies IntegrityChecked;
```

The checker may then close the property set under declared implications. Care must be taken to avoid arbitrary rule systems that make analysis unpredictable or undecidable. Restricting implications to a finite acyclic relation yields a simple and useful feature.

14 Tag Transformers

Many functions systematically transform tags. Instead of enumerating contracts for every user tag, generic transformers could be declared.

```
function identityLike<T>(T x)
    preserves all value_tags(x);
```

More cautiously:

```
function immutableCopy<T>(T x)
    preserves tags classified ValueProperty;
```

A sanitiser may replace one tag by another:

```
function sanitizeHtml(s)
    consumes s UntrustedText;
    ensures result HTMLSafe;
```

This makes the property system useful as an extensible data-flow protocol.

15 Negative Information

Negative tags require explicit semantics. `!Authenticated` should mean *definitely not authenticated*, rather than merely *not known to be authenticated*. This distinction prevents unsound reasoning from absence of evidence.

For every tag T, the checker therefore distinguishes:

$$\begin{aligned} T(v) &\quad \text{proved,} \\ \neg T(v) &\quad \text{proved,} \\ ?T(v) &\quad \text{unknown.} \end{aligned}$$

At a join between a path proving T and a path proving $\neg T$, the result is unknown. A contradictory state containing both T and $\neg T$ should be unreachable or reported as an internal inconsistency in annotations/contracts.

16 Trust Boundaries and Proof Sources

A static checker is only as meaningful as the mechanism by which facts enter the system. We distinguish several sources of property evidence:

1. **Compiler-proven facts.** For example, a bounds check may permit the compiler to establish `InBounds`.
2. **Branch refinements.** A recognised predicate refines properties on true/false branches.
3. **Verified function contracts.** The callee body is checked against its declared postconditions.
4. **Externally trusted contracts.** Native code, libraries, generated code, or APIs may be annotated but not locally verified.
5. **Explicit assumptions.** The programmer intentionally bypasses proof.

Compilers should make these categories visible. A diagnostic may say not merely that a property is present, but that it ultimately depends on an unchecked assumption at a particular source location.

This supports a useful notion of a *trust budget*: large systems can audit the relatively small set of assumption points while relying on automatic propagation elsewhere.

17 Diagnostics and Proof Traces

A practical checker should produce explanatory diagnostics. Instead of:

```
error: missing tag Validated
```

it should report something closer to:

```
error: cannot prove Validated(payment) here
  property established at line 42 on branch condition verify(payment)
  but control may reach this call through predecessor line 57
  where payment was reassigned at line 53
```

Optionally, the compiler could expose a compact proof trace describing how each required tag was established. This would substantially improve usability and make the system attractive as a code-review aid.

18 Relationship to Existing Techniques

The proposal overlaps with several established ideas but occupies a distinct pragmatic design point.

18.1 Taint analysis

Taint systems propagate security labels such as *untrusted* and *sanitised*. User-defined static tags generalise this idea to arbitrary domain facts rather than a predefined security lattice. A sufficiently expressive taint framework can encode much of the proposal, but mainstream languages rarely expose taint tracking as a universal first-class semantic annotation system.

18.2 Typestate

Typestate associates legal operations with an object's abstract state, such as `Open` versus `Closed`. Static tags can encode typestate but are more orthogonal: a value may simultaneously possess many independent properties rather than inhabit one enumerated protocol state.

18.3 Refinement types

A refinement type may express a predicate such as

$$\{x : \mathbb{Z} \mid x > 0\}.$$

Property tags can be viewed as named, opaque refinements whose proof rules are explicitly supplied by data-flow events and contracts. This is less expressive but potentially easier to adopt and cheaper to analyse.

18.4 Dependent types

Dependent type systems allow types to depend on values and can encode rich invariants with explicit or synthesised proofs. The present proposal deliberately separates ordinary machine-level type from lightweight semantic knowledge. One might regard it as a restricted proof-carrying layer in which propositions are user-named atoms and proof composition is primarily data-flow propagation.

18.5 Effect systems

Effect systems annotate expressions or functions with computational effects. Property tags concern facts about values, but the machinery is complementary. A language might track both function effects and value properties within a unified static contract framework.

18.6 Abstract interpretation

From an implementation perspective, the checker is naturally described as an abstract interpretation over a finite property domain. The distinctive language-design contribution is to expose the abstract domain to users through extensible symbolic properties and source-level proof obligations.

19 Comparison with Wrapper Types

Many teams encode semantic state with wrapper types:

```
class ValidatedString { String value; }
class SqlSafeString  { String value; }
```

This is often effective, but has limitations. It may create allocation or representation concerns, wrapper proliferation, cumbersome conversions, and difficulty representing multiple orthogonal facts. Tags allow:

```
String s;   // ordinary representation
// statically known: Validated, Normalized, SQLSafe
```

without synthesising a combinatorial family of wrapper types. Conversely, wrapper types provide stronger nominal abstraction and may be preferable when semantic states correspond to genuinely distinct APIs. The two mechanisms are complementary.

20 Security Applications

The proposal is particularly well suited to security-relevant flow constraints:

- untrusted versus validated input;
- authentication and authorisation state;
- context-specific escaping;
- signature verification;
- cryptographic key provenance;
- secrets that must not reach logging sinks;
- declassification and endorsement boundaries;
- deserialised objects whose invariants have been checked.

A security policy could require:

```
function log(x)
    requires x !Secret;
```

or, more safely, use an information-flow lattice in an advanced extension.

21 Numerical and Scientific Applications

The same mechanism is useful outside security. Examples include:

```
establish matrix Symmetric;
establish matrix PositiveDefinite;
establish portfolio DeltaNeutral;
establish vector Normalized;
establish timestamp Monotonic;
establish quantity UnitChecked;
establish mesh ManifoldChecked;
```

A numerical routine could state:

```
function cholesky(A)
    requires A Symmetric PositiveDefinite;
```

or a market-risk routine:

```
function publishHedgeBook(book)
    requires book DeltaNeutral RiskLimitsChecked;
```

These tags need not prove the underlying mathematics themselves. Rather, they record that designated verification stages have occurred and prevent subsequent code from accidentally bypassing them.

22 Temporal and Protocol Applications

Tags can record protocol milestones:

```
establish socket Connected;
establish message Parsed;
establish request Authenticated;
establish transaction Committed;
```

Unlike classical typestate, independent properties may coexist. This is convenient for systems in which state is not naturally a single finite-state machine.

23 Capability-Like and Linear Tags

An extension could permit tags whose duplication is restricted. For example:

```
establish token HasWriteCapability(resource42);
```

If the capability tag is declared *linear*, copying the underlying value would not duplicate authority. Instead the property would move from source to destination. This connects the proposal to affine and linear type systems and could support resource ownership, single-use tokens, and lifecycle protocols.

This feature should be considered an optional extension because it changes the simple set-propagation semantics of ordinary tags.

24 Soundness Considerations

A core soundness theorem can be stated informally as follows.

Property 1 (Static requirement soundness)**.** *Assume that all tag-introducing primitives and function contracts are semantically valid, that the analysis soundly over-approximates control flow and aliasing, and that no unchecked assumption is used. If the checker accepts* `require x T` *at program point p, then every concrete execution reaching p satisfies the semantic predicate associated with T for the runtime value denoted by x.*

The theorem deliberately depends on the validity of primitive tag introduction and contracts. Opaque tags do not by themselves create truth; they create a tractable framework for transporting established truth.

Unchecked `assume` operations should therefore be treated similarly to unsafe casts, foreign-function interfaces, or proof axioms: useful but explicitly visible.

25 Completeness and False Positives

The proposed analysis is not expected to be complete. A property may hold at runtime even when the checker cannot prove it. Conservative rejection is preferable to unsound acceptance for safety-oriented uses.

Developers can resolve false positives through:

- stronger function contracts;
- explicit branch refinements;
- loop invariants;
- local restructuring;
- trusted assumptions where justified;
- advanced analysers that infer more preservation facts.

The design should avoid turning every rejected proof obligation into a demand for complex theorem proving.

26 Gradual Adoption

A major advantage of opaque symbolic tags is that they can be introduced gradually. Existing code remains valid until a new `require` or contract is added. A possible adoption sequence is:

1. annotate critical sink functions with requirements;
2. annotate existing validators with postconditions;
3. run the checker and identify unproved flows;
4. add missing contracts or explicit assumptions at legacy boundaries;
5. progressively reduce assumption points.

This resembles the successful adoption patterns of nullability annotations, static taint checking, and gradual typing.

27 Implementation Strategy

A prototype could be implemented as a compiler or linter phase using standard data-flow infrastructure.

27.1 Front-end representation

The source compiler lowers code to a control-flow graph, preferably SSA. Each SSA value is associated with an abstract property environment.

27.2 Worklist algorithm

For each basic block, the checker maintains an input and output abstract state. Transfer functions process statements; predecessor states are joined by intersection of definite positive and negative facts. A standard worklist iterates until no state changes.

If the tag universe present in a compilation unit is finite, termination is immediate because the lattice has finite height.

27.3 Interprocedural analysis

The simplest implementation trusts declared contracts. A stronger implementation verifies function bodies against `ensures` clauses and computes summaries for unannotated functions. Whole-program or modular summary inference may reduce annotation burden.

27.4 Aliasing

For pointer-rich languages, the checker can reuse existing alias, escape, ownership, or memory-SSA infrastructure. A deliberately conservative first prototype may invalidate all object-state properties after unknown impure calls.

27.5 Runtime cost

The basic system has zero runtime cost: tags and requirements are erased after compilation. Optional debug builds may translate selected requirements into runtime assertions, but this is not necessary to the design.

28 A Compact Formalisation

Let a statement transform abstract state Σ to $F_s(\Sigma)$. Representative rules are:

Establish.

$$F_{\text{establish}(v,t)}(\Sigma): \quad t \in P^+(v),\ t \notin P^-(v).$$

Refute.

$$F_{\text{refute}(v,t)}(\Sigma): \quad t \in P^-(v),\ t \notin P^+(v).$$

Forget.

$$F_{\text{forget}(v,t)}(\Sigma): \quad t \notin P^+(v),\ t \notin P^-(v).$$

Require.

$$\Sigma \vdash \text{require}(v,t) \quad \text{iff} \quad t \in P^+(v).$$

Negative require.

$$\Sigma \vdash \text{require}(v,!t) \quad \text{iff} \quad t \in P^-(v).$$

Join. For predecessor states Σ_1 and Σ_2:

$$\text{join}(P_1^+, P_2^+) = P_1^+ \cap P_2^+, \qquad \text{join}(P_1^-, P_2^-) = P_1^- \cap P_2^-.$$

This minimal semantics already provides useful correctness checking.

29 Example: Authentication Protocol

Consider:

```
Request r = receive();

if (!verifySignature(r))
    reject();
// Contract refines success continuation:
// r has SignatureVerified

User u = lookupUser(r);

if (!authenticate(u, r.credentials))
    reject();
// u has Authenticated

Payment p = parsePayment(r);
require r SignatureVerified;

if (!authorise(u, p))
    reject();
// p has AuthorisedFor(u)

submit(p);
```

with contracts:

```
function submit(p)
    requires p Authorised;
    requires p Validated;
```

The checker enforces that code cannot accidentally bypass one of the protocol phases even when functions are reorganised or control flow becomes complex.

30 Example: Representation-Sensitive Validation

Suppose a compressed payload is validated before decompression:

```
Blob compressed = receive();
validateContainer(compressed);
establish compressed ContainerValid;

Blob plain = decompress(compressed);
require plain ContainerValid;  // rejected
```

The rejection forces a semantic question that is easy to miss during ordinary code review: does validity of the container imply validity of the decompressed content? Usually it does not. The correct design might instead specify:

```
contract decompress(x)
    requires x ContainerValid;
    ensures result Decompressed;
```

followed by a separate content validation phase.

31 Example: Financial Correctness Workflow

A static property system can encode workflow invariants in quantitative software:

```
Book b = loadTrades();
reconcile(b);
establish b Reconciled;

computeGreeks(b);
establish b GreeksCurrent;

hedge(b);
establish b DeltaNeutral;

publishRisk(b);
```

where:

```
function publishRisk(Book b)
    requires b Reconciled GreeksCurrent DeltaNeutral;
```

If a later mutation inserts a trade after hedging, contracts can invalidate `GreeksCurrent` and `DeltaNeutral`, statically preventing stale state from being published.

32 Automatic Tag Inference

Although user-defined tags are intentionally opaque, a compiler can infer some tag changes from recognised constructs. Examples include:

- bounds checks establishing `InBounds`;
- null tests establishing `NonNull`;
- lock acquisition establishing `LockHeld(L)`;
- successful parse operations establishing `Parsed`;
- enum discrimination establishing variant-specific facts;
- ownership operations establishing uniqueness properties.

A plugin architecture could allow domain-specific analysers to contribute proof rules while keeping the core language simple.

33 Proof-Carrying Validators

A deeper extension replaces an imperative tag-introduction statement with validators that return a proof token:

```
Proof<Validated(x)> p = validate(x);
```

The compiler may erase the token at runtime. This begins to resemble dependent typing, but a mainstream language could hide most proof syntax and allow the result merely to refine tags. Such proof objects would make trust dependencies explicit and might enable verified libraries to interoperate with the lightweight tag layer.

34 Tag Scope and Revocation

Some facts are valid only during a lexical or dynamic scope. For example, a lock is held only within a region:

```
with lock(m) {
    // m has LockHeld
    require m LockHeld;
}
// LockHeld no longer available
```

Scoped tags are useful for locks, transactions, temporary pinning, privilege elevation, borrowed views, and read-consistency snapshots. The language can model scope exit as automatic invalidation.

35 Concurrency

Concurrency complicates object-state tags. If another thread may mutate an object, properties established by one thread can become stale. A sound design requires one or more of:

- immutability;
- thread confinement;
- ownership or race-freedom guarantees;
- lock-conditioned properties such as `ValidUnder(lock)`;
- conservative invalidation at concurrency boundaries.

This suggests a productive integration with race detectors, ownership systems, or effect annotations.

36 Exceptions and Non-Local Control Flow

Property propagation must follow all exceptional exits. If a function establishes a tag only after an operation that can throw, the fact is available only on normal continuation. Finally blocks, destructors, cancellation, and asynchronous control flow require ordinary CFG treatment.

The mechanism itself introduces no fundamentally new problem; it inherits the compiler's control-flow model.

37 Generics and Polymorphism

A useful extension is property polymorphism. Consider an identity function:

```
T id<T>(T x)
    preserves value_properties(x);
```

or a container mapper that preserves element properties only when the mapping function does:

```
map(xs, f)
    propagates ElementTag(P)
    if f preserves P;
```

Such features should be introduced cautiously. Even a non-polymorphic first version offers substantial value.

38 Design Principle: Keep the Logic Small

The primary risk is uncontrolled evolution into a theorem-proving language. The proposed system is most attractive if its default fragment remains predictable:

- finite user-defined atomic tags;
- three-valued knowledge;
- flow-sensitive propagation;
- explicit contracts;
- finite implication relations;
- conservative invalidation under unknown mutation.

Richer refinements, arithmetic predicates, SMT solving, linear capabilities, and proof terms can exist as optional layers rather than prerequisites.

39 Potential Syntax in a Mainstream Language

One Java-like design might use annotations and compiler intrinsics:

```
@Requires("Validated", "Authorised")
void submit(Payment p) { ... }

@Ensures(param="p", tag="Validated")
boolean validate(Payment p) { ... }

if (!validate(p)) return;
Static.require(p, "Validated");
```

A language with first-class syntax could be cleaner:

```
fn submit(p: Payment)
    requires p: Validated + Authorised
{
    ...
}

fn validate(p: Payment) -> bool
    true => ensures p: Validated
{
    ...
}
```

For backward compatibility, an implementation could initially support the feature entirely through attributes and a compiler plug-in.

40 Research Questions

The proposal suggests several empirical and theoretical research questions:

1. How much annotation is required to obtain useful guarantees in large legacy codebases?
2. Which default preservation rules minimise both false positives and unsound intuition?
3. Can useful contracts be inferred automatically from common validator patterns?
4. How should tags interact with aliasing, ownership, and concurrency?
5. Are developers able to understand three-valued property states and proof diagnostics without specialist training?
6. Does the mechanism reduce classes of real defects compared with wrapper types, assertions, or conventional linters?
7. What is the optimal boundary between opaque tags and parameterised/refinement-like properties?
8. Can assumption provenance be surfaced effectively enough to support security audits and certification?

41 Prototype Evaluation Plan

A reasonable prototype study could proceed as follows:

1. Implement an analyser over a mainstream language IR such as LLVM, Java bytecode/AST, Roslyn C#, or Clang C++.
2. Support atomic positive/negative tags, branch refinement, requirements, and function contracts.
3. Evaluate on three domains: input validation/security, protocol/stateful APIs, and numerical/financial workflows.
4. Measure annotation density, false-positive rate, compile-time overhead, and defects detected.
5. Compare against wrapper-type refactorings and dedicated taint analysers.
6. Conduct a developer study focused on diagnostic comprehensibility and perceived burden.

42 Limitations

The mechanism does not eliminate the need for testing, runtime validation, formal verification, or domain reasoning. Opaque tags can record that a validation step occurred, but a buggy validator can establish a false belief. Incorrect contracts can likewise compromise guarantees. Aliasing and concurrency may force conservative information loss. Furthermore, properties requiring nontrivial arithmetic or relational reasoning are only indirectly representable unless the system is extended toward refinement types or SMT-backed verification.

Nevertheless, these limitations are shared in various forms by most practical static-analysis systems. The value proposition is not complete program verification, but a low-friction way to make semantic correctness obligations explicit and machine-checkable.

43 Conclusion

User-defined static property tags provide a plausible lightweight verification layer for mainstream programming languages. They generalise familiar ideas from type narrowing and taint analysis into an extensible mechanism for recording semantic facts about values. The essential design is simple: tags are introduced at justified proof points, propagated by flow-sensitive analysis, invalidated by transformations that may destroy their meaning, and demanded by compile-time requirements and function contracts.

The proposal occupies a useful middle ground. Ordinary type systems often cannot express whether a value has been validated, authorised, normalised, reconciled, range checked, cryptographically verified, or otherwise processed according to a semantic protocol. Full refinement or dependent type systems can express substantially more, but at greater conceptual and implementation cost. Static property tags intentionally trade expressive completeness for predictable data-flow reasoning and gradual adoption.

The most important engineering questions concern value identity, aliasing, mutation, interprocedural preservation, trust boundaries, and diagnostic quality. These problems are tractable using established compiler techniques such as SSA, abstract interpretation, points-to analysis, function summaries, and fixed-point data-flow algorithms. With careful language design, the feature could provide many of the practical correctness benefits associated with richer type systems while remaining intelligible to programmers accustomed to conventional mainstream languages.

In this sense, the proposed mechanism can be understood as a lightweight, programmer-extensible layer of atomic propositions over program values: a restricted proof system whose proofs are largely supplied by control flow, provenance, and contracts rather than by explicit theorem construction. That restrictedness may be its principal strength.

Suggested Related Literature

The proposal is conceptually connected to a substantial body of existing work. A future publication should position it carefully with respect to refinement types, typestate, effect systems, abstract interpretation, security-typed languages, taint analysis, liquid types, gradual typing, ownership and linear types, and dependent types. Canonical starting points include work by Cousot and Cousot on abstract interpretation, Strom and Yemini on typestate, Freeman and Pfenning on refinement types, and later work on Liquid Types and practical refinement checking. Exact bibliographic references should be selected and verified for the target venue.